王秉亨

RESEARCH ASSESSMENT / REVISED EDITION

Differentiable MHE: Structural Originality and Recursive Solvers

An equation-level assessment of DMHE and NeuroMHE against fast MHE, Riccati decomposition, sparse matrix factorization, and constrained auxiliary sensitivity optimization

Central result. The 2008 Riccati and NeuroMHE Kalman matrix recursions admit an algebraic mapping, but the complete conversion to a weighting-parameter *gradient filter* is non-trivial. NeuroMHE constructs and proves an auxiliary sensitivity-MHE correspondence absent from the 2008 paper. Its auxiliary P_k, C_k are *not* the original estimator's EKF arrival-cost update.

This edition addresses

- Exact term-by-term transformation of Zavala et al. (2008), Eqs. (12)–(14), into NeuroMHE, Eqs. (17)–(20).
- Original-MHE versus auxiliary-MHE objective comparison: a *shared arrival weight* P_a but *different stagewise weights* (noise precisions versus Lagrangian Hessians), and the meaning of $P_{t-N}^{\mathrm{KF}} = P_a^{-1}$.
- The solver flexibility, interpretability, and constrained extensibility supplied by auxiliary optimization.
- Fully separated operation costs: explicit inversion, dense and sparse LU, Riccati, Kalman recursions, and constrained QP algorithms.

Basis: User-supplied journal/conference papers and the supplied forward-DP/arrival-cost technical report; mathematical transformations derived and labeled separately.

Prepared: 27 October 2026 **Scope:** Scientific originality and numerical methodology

Runtime update (29 October 2026): sparse KKT assembly, symbolic reuse, and benchmark evidence in Sec. 13.

Contents

1 Executive findings and evidence boundaries **3**

1.1 Three levels of comparison . . . 3

2 Common mathematical setting and source notation **4**

2.1 One consistent stage notation . . . 4

3 Exact coefficient-level comparison of the forward recursions **5**

3.1 Original fast-MHE KKT stage equations . . . 5

3.2 Modified transition and local information terms . . . 5

3.3 Initial reduced matrix . . . 5

3.4 Forward predicted matrix . . . 5

3.5 Forward filtered/reduced matrix . . . 6

4 Affine RHS, arrival sensitivity, and Kalman prediction/correction **6**

4.1 The sensitivity RHS is not the ordinary Newton residual . . . 6

4.2 Affine predictor: transforming fast MHE Eq. (14) . . . 6

4.3 Affine correction: recovering NeuroMHE Eq. (18d) . . . 7

5 Backward trajectories and the smoothing interpretation **7**

5.1 State reconstruction (fast MHE Eq. (12c)) . . . 7

5.2 Multiplier reconstruction (fast MHE Eq. (12b)) . . . 8

5.3 Process-noise recovery (fast MHE Eq. (12d)) . . . 8

5.4 Terminal recovery (fast MHE Eq. (12a)) . . . 8

6 Covariance correction: original MHE versus auxiliary sensitivity MHE **9**

6.1 What Zavala et al. actually do in Section 3.3 . . . 9

6.2 What NeuroMHE computes instead . . . 9

6.3 A direct explanation from the two objective functions . . . 10

6.4 Why the initial inverse arrival weight does not equate the two covariance roles . . . 11

6.5 The different update laws, and a possible special-case numerical match . . . 12

7 Is the conversion trivial, and what is original? **14**

7.1 Degree of difficulty of each transformation . . . 14

7.2 Prior art versus novel presentation and capability . . . 14

8 Full numerical operation counts and wall-clock scope **15**

8.1 Explicit inverse: what the historical exponent describes 15

8.2 Conventional general-purpose dense LU 15

8.3 Sparse LU or $LDL^\top$ on bounded-stage chain KKT 15

8.4 Classical Riccati versus forward Kalman sensitivity recursion 15

8.5 Consolidated complexity table 16

9 Why Riccati and ordered sparse elimination are algebraically related **16**

10 The methodological merits of auxiliary optimization **17**

10.1 Constructing the sensitivity problem without numerical KKT factorization 17

10.2 Solver choice is a feature, not a new Riccati identity 17

10.3 Soft/barrier treatment is not exact hard-constrained differentiation 17

10.4 Weakly active constraints yield a genuinely constrained directional sensitivity problem . 18

10.5 Computational consequence for inequality-constrained QPs 19

11 Positioning against differentiable-control and constrained-MHE prior art **19**

12 Final assessment and recommended wording **20**

13 Practical runtime accounting and solver interfaces **21**

13.9 From stagewise derivative blocks to the KKT solve 26

Source documents and references **28**

1. Executive findings and evidence boundaries

Bottom line. The fast-MHE paper of Zavala, Laird, and Biegler [3] already gives a forward Riccati decomposition of the MHE KKT/Newton system and a general NLP sensitivity equation. Under matching KKT coefficients and right-hand sides, its stagewise matrix and recovery operations can be mapped algebraically to the NeuroMHE sensitivity solver. *It does not itself explicitly construct an auxiliary matrix-valued parameter-sensitivity MHE or derive the Kalman-form weighting-gradient recursion.*

Finding 1—matrix equivalence without identity of presentation. With common stage Hessians, dynamics Jacobians, and boundary data, the fast-MHE matrices Π_k and M_k coincide with NeuroMHE's filtered and predicted matrices C_k and P_k^{KF}. The affine and recovery recursions require additional substitutions. The mapping explains the shared numerical linear algebra; it does not show that the 2008 authors explicitly derived the gradient Kalman algorithm.

Finding 2—different covariance roles, potentially matching coefficients. Fast MHE Section 3.3 extracts covariance/curvature information for the *original MHE estimator* and its next arrival cost. NeuroMHE's P_k, C_k are coefficient matrices of an *auxiliary sensitivity filter*; its $\hat{X}^{\mathrm{KF}}$ is a parameter-gradient trajectory. The original and auxiliary MHE *arrival objectives use the same information weight* P_a, but their *running costs differ*: NeuroMHE uses Lagrangian Hessians and mixed parameter derivatives. This explains $P_{t-N}^{\mathrm{KF}} = P_a^{-1}$ without confusing auxiliary matrices with a physical arrival covariance. Values can coincide for matched linear-quadratic problems; purposes and outputs still differ. See sections 6 and 6.3.

Finding 3—structural novelty. DMHE [1] and NeuroMHE [2] explicitly identify parameter gradients with the optimum of a matrix-valued LQ estimation problem, prove the differential-KKT/auxiliary-KKT correspondence, and derive a Kalman filtering–smoothing calculation. This is a substantive *formulation and interpretation* relative to [3], without claiming first-ever $O(N)$ MHE linear-system solution.

Finding 4—numerical costs. Dense LU has $O(N^3)$ factorization and $O(N^2 p)$ triangular solutions for p right-hand sides; suitable ordered sparse LU and Riccati/Kalman methods can each scale linearly in N at fixed stage dimensions. The historical $O(N^{2.376})$ bound pertains to fast dense linear algebra, not a lower bound for solving structured KKT equations. See section 8.

Finding 5—auxiliary systems are broader than an individual solver. They expose the objective, stagewise dynamics, boundary forcing, and derivative constraints, enabling principled solver selection. For weakly active inequalities, an appropriate auxiliary *directional sensitivity* QP can retain inequalities; a standalone linear solve does not resolve its complementarity decisions. This is prior art in Das and Jäschke (2017) [6], not a demonstrated hard-inequality capability of the published NeuroMHE Kalman recursion.

Finding 6—explicit originality judgment. Transforming fast MHE Eqs. (12)–(14) into the *complete* NeuroMHE Eqs. (17)–(20) is **not a trivial substitution**. Matrix-level Schur-complement identities are reusable prior art, but the parameter-sensitive affine forcing, inherited arrival-center derivative, Kalman prediction/correction variables, multiplier smoothing, and auxiliary-MHE optimality proof require additional mathematical construction. These furnish a substantive methodological contribution relative to the cited 2008 work; they do not constitute a new fundamental Riccati identity.

1.1 Three levels of comparison

1. **Same numerical problem:** Does each algorithm solve the same linearized KKT equations with the same Hessian approximation and right-hand side?
2. **Same numerical operations:** Is there an algebraic change of variables relating its coefficient recursions and recovery steps?

3. **Same conceptual contribution:** Did the earlier work explicitly formulate an auxiliary sensitivity optimization problem and interpret its solution as a Kalman-filtered/smoothed parameter-gradient trajectory?

Only the first two follow from the derivations below. The third must be answered directly from what the cited papers actually present.

2. Common mathematical setting and source notation

Let the original MHE minimize a sum of an arrival penalty and stage penalties, subject to $z_{k+1} = f_k(z_k, w_k)$. Its locally optimal state, process-noise, and dynamics-multiplier trajectories depend on a parameter $\theta \in \mathbb{R}^p$ (e.g., learned weighting parameters). At the solved trajectory define

$$X_k = \frac{\partial z_k^*}{\partial \theta}, \qquad W_k = \frac{\partial w_k^*}{\partial \theta}, \qquad \Lambda_k = \frac{\partial \lambda_k^*}{\partial \theta}. \tag{1}$$

For a single parameter direction $d\theta$, all these are vectors; the paper's matrix-valued implementation stacks directions into columns. Differentiation of the first-order optimality conditions gives

$$\begin{bmatrix} H & D^\top \\ D & 0 \end{bmatrix} \begin{bmatrix} X \\ \Lambda \end{bmatrix} = - \begin{bmatrix} B_{\text{stat}} \\ B_{\text{dyn}} \end{bmatrix}, \tag{2}$$

with Hessian H, dynamics Jacobian D, stationarity forcing B_{stat}, and dynamics forcing B_{dyn}. This is a formal implicit derivative; it does not mandate explicit inversion of the coefficient matrix.

2.1 One consistent stage notation

The two publications use the same symbols for different objects. The following convention will be used in the derivation:

This report	Fast MHE [3]	NeuroMHE [2]	Meaning
Q_k	P_k	L_k^{xx}	State-state Hessian
D_k	F_k	L_k^{xw}	State–noise cross Hessian
R_k	W_k	L_k^{ww}	Noise–noise Hessian
A_k	A_k	F_k	Dynamics state Jacobian
G_k	G_k	G_k	Dynamics noise Jacobian
Π_k	Π_k	C_k	Reduced/filtered matrix
M_k	M_k	P_k	Predicted matrix
$r_{z,k}, r_{w,k}$	$r_{z,k}, r_{w,k}$	$L_k^{x\theta}, L_k^{w\theta}$, plus arrival term	RHS forcing

The comparison is specialized to *parameters in the original MHE objective/weightings*, as in NeuroMHE: its model dynamics are parameter-independent. Therefore $B_{\text{dyn}} = 0$. For general trainable dynamics, extra dynamics-parameter forcing must be retained and the simple zero-forcing Kalman prediction formula changes.

Assume stage R_k and the Schur complements used below are invertible. Write $b = t - N$ for the initial stage, shift it to 0 for the derivation, and index subsequent stages by $k = 1, \ldots, N$. The original arrival penalty has an information weight $P_a \succ 0$, and its center may itself depend on θ. Consequently

$$Q_0 = P_a + \bar{Q}_0, \qquad Q_k = \bar{Q}_k \ (k > 0), \qquad \bar{X} := \frac{\partial \hat{z}_{\text{arr}}}{\partial \theta}. \tag{3}$$

These are the *original differentiable MHE objective*'s second derivatives. They are not being asserted to be stochastic Kalman covariances.

3. Exact coefficient-level comparison of the forward recursions

3.1 Original fast-MHE KKT stage equations

Zavala et al. [3, Sec. 3.1, Eqs. (9a)–(9e)] present the Newton/search-direction variables $(\Delta z_k, \Delta w_k, \Delta\lambda_{k+1})$ satisfying

$$Q_k\Delta z_k + D_k\Delta w_k + \Delta\lambda_k - A_k^\top\Delta\lambda_{k+1} = -r_{z,k}, \tag{4}$$

$$D_k^\top\Delta z_k + R_k\Delta w_k - G_k^\top\Delta\lambda_{k+1} = -r_{w,k}, \tag{5}$$

$$\Delta z_{k+1} - A_k\Delta z_k - G_k\Delta w_k = -r_{\lambda,k+1}. \tag{6}$$

At $k = 0$ the incoming multiplier term is absent; at $k = N$ there is no outgoing dynamics or noise variable. For parameter-independent dynamics, $r_{\lambda,k+1} = 0$ in the sensitivity solve. NeuroMHE's differentiated KKT equations, Eq. (13), have the same stagewise form after identifying $\Delta z_k = X_k$, $\Delta w_k = W_k$ and shifting the indexing of $\Delta\lambda_{k+1}$ to Λ_k.

The subtle but essential distinction: the fast-MHE equations are presented *first* as a Newton system. They become parameter sensitivities only after substituting the parameter-derivative RHS supplied by the fast-MHE general NLP sensitivity relation, Eq. (16); at the converged optimum, merely reusing the zero Newton residual would give a zero step.

3.2 Modified transition and local information terms

Eliminate the noise variable from (5). Introduce

$$\bar{A}_k := A_k - G_kR_k^{-1}D_k^\top, \tag{7}$$

$$S_k := D_kR_k^{-1}D_k^\top - \bar{Q}_k, \tag{8}$$

$$T_k := D_kR_k^{-1}r_{w,k} - r_{z,k}^{\text{base}}. \tag{9}$$

Here $r_{z,k}^{\text{base}} = L_k^{x\theta}$; the arrival-center variation is separated out at stage 0. At the final stage $S_N = -Q_N$, $T_N = -r_{z,N}$. These are precisely the $\bar{F}_k$, S_k, and T_k coefficients in NeuroMHE's Eqs. (17)–(20), after the notation table in section 2 is applied.

3.3 Initial reduced matrix

Fast MHE Eq. (13) starts from

$$\Pi_0 = (Q_0 - D_0R_0^{-1}D_0^\top)^{-1}. \tag{10}$$

NeuroMHE, Eq. (17a)–(17b), sets the auxiliary prediction matrix $P_0^{\text{KF}} = P_a^{-1}$ and

$$C_0 = (I - P_0^{\text{KF}}S_0)^{-1}P_0^{\text{KF}} = (P_a - S_0)^{-1}. \tag{11}$$

By (3) and (8),

$$C_0 = (P_a + \bar{Q}_0 - D_0R_0^{-1}D_0^\top)^{-1} = \Pi_0. \tag{12}$$

This transformation requires recognizing that Q_0 contains the arrival-cost Hessian; it is not literally a symbol-for-symbol replacement.

3.4 Forward predicted matrix

Fast MHE Eq. (13), with its two negative factors collected, gives

$$M_{k+1} = \bar{A}_k\Pi_k\bar{A}_k^\top + G_kR_k^{-1}G_k^\top. \tag{13}$$

NeuroMHE Eq. (18b) is

$$P_{k+1}^{\mathrm{KF}} = \bar{A}_k C_k \bar{A}_k^\top + G_k R_k^{-1} G_k^\top. \tag{14}$$

If $\Pi_k = C_k$, then $M_{k+1} = P_{k+1}^{\mathrm{KF}}$.

3.5 Forward filtered/reduced matrix

Fast MHE Eq. (13) also gives, for $1 \le k < N$,

$$\Pi_k = \left(Q_k + M_k^{-1} - D_k R_k^{-1} D_k^\top\right)^{-1}. \tag{15}$$

The NeuroMHE Eq. (18c) update is

$$C_k = (I - P_k^{\mathrm{KF}} S_k)^{-1} P_k^{\mathrm{KF}} = \left((P_k^{\mathrm{KF}})^{-1} - S_k\right)^{-1}. \tag{16}$$

Using (8) and $P_k^{\mathrm{KF}} = M_k$ makes (15) and (16) identical. At $k = N$ the cross Hessian is absent: $\Pi_N = (Q_N + M_N^{-1})^{-1}$ and $S_N = -Q_N$ yield the same identity.

Exact forward matrix correspondence:

$$\boxed{\Pi_k^{\mathrm{Fast}} = C_k^{\mathrm{Neu}}, \qquad M_k^{\mathrm{Fast}} = P_k^{\mathrm{Neu}} \quad (k \ge 1).} \tag{17}$$

This part of the conversion follows by recognizing the common Schur complements. **It is comparatively direct after identifying the Hessian blocks.** It alone does not establish a full parameter-gradient Kalman algorithm. The equality of coefficient matrices also does *not* equate the original MHE arrival-covariance calculation with a Kalman filtering update of parameter-gradient trajectories; these share KKT data but solve conceptually distinct tasks.

4. Affine RHS, arrival sensitivity, and Kalman prediction/correction

4.1 The sensitivity RHS is not the ordinary Newton residual

Write the fast-MHE RHS $r_{z,k}, r_{w,k}, r_{\lambda,k+1}$. For MHE weighting sensitivities, the appropriate substitution is

$$r_{w,k} = L_k^{w\theta}, \qquad r_{z,k} = L_k^{x\theta} \quad (k \ge 1),\ r_{z,0} = L_0^{x\theta} - P_a \bar{X}, \qquad r_{\lambda,k+1} = 0. \tag{18}$$

The $-P_a\bar{X}$ term is from the derivative of the original arrival-center penalty and must not be omitted. NeuroMHE Eq. (13a) includes this contribution explicitly; Appendix A gives an approximation to the inherited $\bar{X}$ from the preceding window.

4.2 Affine predictor: transforming fast MHE Eq. (14)

At the first step, fast MHE Eq. (14), with $r_{\lambda,1} = 0$, becomes

$$r_{M,1} = G_0 R_0^{-1} r_{w,0} + \bar{A}_0 \Pi_0 \left(r_{z,0} - D_0 R_0^{-1} r_{w,0}\right). \tag{19}$$

Define the auxiliary filtered sensitivity at the initial stage as

$$\hat{X}_{0|0}^{\mathrm{KF}} := C_0 (P_a \bar{X} + T_0). \tag{20}$$

Since $T_0 = D_0 R_0^{-1} r_{w,0} - L_0^{x\theta}$, Eqs. (18)–(20) imply

$$r_{M,1} = G_0 R_0^{-1} r_{w,0} - \bar{A}_0 \hat{X}^{\mathrm{KF}}_{0|0}. \tag{21}$$

Introduce the predicted sensitivity by the *change of variable*

$$\boxed{\hat{X}^{\mathrm{KF}}_{k|k-1} := -r_{M,k} \qquad (k \geq 1).} \tag{22}$$

Fast MHE does not introduce $r_{M,k}$ as a filtered sensitivity; (22) is part of the new comparison.

At an interior stage, fast MHE Eq. (14) reads

$$\begin{aligned} r_{M,k+1} &= G_k R_k^{-1} r_{w,k} + \bar{A}_k \Pi_k \big(r_{z,k} + M_k^{-1} r_{M,k} - D_k R_k^{-1} r_{w,k}\big) \\ &= G_k R_k^{-1} r_{w,k} - \bar{A}_k C_k \big(T_k + M_k^{-1} \hat{X}^{\mathrm{KF}}_{k|k-1}\big). \end{aligned} \tag{23}$$

Define

$$\hat{X}^{\mathrm{KF}}_{k|k} := C_k \big(T_k + M_k^{-1} \hat{X}^{\mathrm{KF}}_{k|k-1}\big). \tag{24}$$

Substituting (24) into (23), then applying (22), yields

$$\boxed{\hat{X}^{\mathrm{KF}}_{k+1|k} = \bar{A}_k \hat{X}^{\mathrm{KF}}_{k|k} - G_k R_k^{-1} r_{w,k}.} \tag{25}$$

This is NeuroMHE Eq. (18a) for the corresponding parameter directions.

4.3 Affine correction: recovering NeuroMHE Eq. (18d)

From (16),

$$C_k^{-1} = M_k^{-1} - S_k, \qquad C_k M_k^{-1} = I + C_k S_k, \qquad M_k^{-1} C_k = I + S_k C_k. \tag{26}$$

Therefore (24) becomes

$$\boxed{\hat{X}^{\mathrm{KF}}_{k|k} = (I + C_k S_k) \hat{X}^{\mathrm{KF}}_{k|k-1} + C_k T_k.} \tag{27}$$

This is NeuroMHE Eq. (18d). The initial equation (20) is equivalent to NeuroMHE Eq. (17c), since $C_0 P_a = I + C_0 S_0$.

Why this part is not a trivial transcription. The conversion replaces a *residual-based affine sequence* $r_{M,k}$ by the negative *predicted sensitivity* $\hat{X}^{\mathrm{KF}}_{k|k-1}$, uses the correct parameter-derivative RHS (not the Newton residual), and accounts for inherited arrival-center sensitivity. These steps are absent as an explicit parameter-gradient Kalman derivation in [3].

5. Backward trajectories and the smoothing interpretation

5.1 State reconstruction (fast MHE Eq. (12c))

Fast MHE Eq. (12c) can be written, after using (7) and (9), as

$$\Delta z_k = \Pi_k \bar{A}_k^{\top} \Delta\lambda_{k+1} + \Pi_k \big(T_k - M_k^{-1} r_{M,k}\big). \tag{28}$$

For $k = 0$ the last factor is modified by the arrival forcing $P_a \bar{X}$ rather than $M_0^{-1} r_{M,0}$, and reproduces (20). For $k \geq 1$, use (17), (22), and (24) to obtain

$$\boxed{\Delta z_k = \hat{X}^{\mathrm{KF}}_{k|k} + C_k \bar{A}_k^{\top} \Delta\lambda_{k+1}.} \tag{29}$$

Identify the multiplier indices $\Lambda_k^* = \Delta\lambda_{k+1}$ (fast MHE indexes the constraint multiplier by the arrival stage, while NeuroMHE indexes it by the departure stage). Then (29) is NeuroMHE Eq. (20):

$$\boxed{\hat{X}_{k|t} = \hat{X}^{\mathrm{KF}}_{k|k} + C_k \bar{F}_k^\top \Lambda_k^*.} \tag{30}$$

The terminal stage has no outgoing dynamics multiplier; NeuroMHE defines $\Lambda_t^* = 0$, which gives

$$\boxed{\hat{X}_{t|t} = \hat{X}^{\mathrm{KF}}_{t|t}.} \tag{31}$$

The *filtered plus smoothed correction* structure is classical for linear Gaussian *state estimation* (e.g., Cox [5]). The novel, source-specific interpretation is its explicit derivation for *MHE parameter sensitivities through an auxiliary matrix-valued estimation problem*. See also section 6 for the unrelated original-arrival covariance.

5.2 Multiplier reconstruction (fast MHE Eq. (12b))

For $1 \le k \le N$, Eq. (12b) gives

$$\Delta\lambda_k = M_k^{-1}(\Delta z_k + r_{M,k}) = M_k^{-1}(\Delta z_k - \hat{X}^{\mathrm{KF}}_{k|k-1}). \tag{32}$$

Substitute (29) and apply (26):

$$\begin{aligned}\Delta\lambda_k &= M_k^{-1}\big(\hat{X}^{\mathrm{KF}}_{k|k} - \hat{X}^{\mathrm{KF}}_{k|k-1}\big) + (I + S_k C_k)\bar{A}_k^\top \Delta\lambda_{k+1} \\ &= S_k \hat{X}^{\mathrm{KF}}_{k|k} + T_k + (I + S_k C_k)\bar{A}_k^\top \Delta\lambda_{k+1}.\end{aligned} \tag{33}$$

With $\Lambda_{k-1}^* = \Delta\lambda_k$, (33) is NeuroMHE Eq. (19). The terminal step is obtained from $S_N = -Q_N$, $T_N = -r_{z,N}$ and the terminal state stationarity condition.

5.3 Process-noise recovery (fast MHE Eq. (12d))

Equation (12d) is

$$\Delta w_k = R_k^{-1}\big(G_k^\top \Delta\lambda_{k+1} - D_k^\top \Delta z_k - r_{w,k}\big). \tag{34}$$

Under the correspondence,

$$\boxed{\hat{W}_k = (L_k^{ww})^{-1}\left(G_k^\top \Lambda_k^* - L_k^{wx}\hat{X}_{k|t} - L_k^{w\theta}\right),} \tag{35}$$

which is NeuroMHE Appendix C, Eq. (36).

5.4 Terminal recovery (fast MHE Eq. (12a))

Fast MHE's terminal formula is

$$\Delta z_N = -\Pi_N\big(r_{z,N} + M_N^{-1} r_{M,N}\big). \tag{36}$$

Since $T_N = -r_{z,N}$ and $\hat{X}^{\mathrm{KF}}_{N|N-1} = -r_{M,N}$, Eq. (36) equals $\hat{X}^{\mathrm{KF}}_{N|N}$ as computed by the terminal version of (24). It is compatible with (31).

Scope of the term-by-term result. Eqs. (17), (25), (27), (30), (33), and (35) establish an algebraic mapping of the *complete linear-system computation*, for identical differential KKT coefficients, forcing, boundary treatment, and invertible stage operations. It is a retrospective transformation of an earlier Newton/KKT Riccati scheme, *not evidence that the 2008 paper explicitly formulated the auxiliary weighting-gradient Kalman filter.*

6. Covariance correction: original MHE versus auxiliary sensitivity MHE

Mandatory distinction. *Fast MHE Section 3.3, Eqs. (21)–(23), is about covariance extraction for the original MHE estimator and its next arrival cost. Fast MHE Section 4, Eqs. (24)–(25), gives a measurement-driven correction of the terminal state estimate. Neither presents the NeuroMHE Kalman-filter-based gradient algorithm.*

6.1 What Zavala et al. actually do in Section 3.3

Fast MHE extracts the local covariance-related matrices from its original estimator KKT/Riccati coefficients. After neglecting nonlinear second-order and state–noise cross contributions, the initial reduced matrix becomes

$$\Pi_0^{\text{orig}} = \left((P_{\text{arr}}^{\text{orig}})^{-1} + H_0^\top R_{\text{meas},0}^{-1} H_0\right)^{-1}. \tag{37}$$

By the matrix inversion lemma,

$$\Pi_0^{\text{orig}} = P_{\text{arr}}^{\text{orig}} - P_{\text{arr}}^{\text{orig}} H_0^\top \left(H_0 P_{\text{arr}}^{\text{orig}} H_0^\top + R_{\text{meas},0}\right)^{-1} \times H_0 P_{\text{arr}}^{\text{orig}}. \tag{38}$$

The next covariance is

$$M_1^{\text{orig}} = A_0 \Pi_0^{\text{orig}} A_0^\top + G_0 Q_{\text{noise},0} G_0^\top. \tag{39}$$

The following MHE window uses $P_{\text{arr}}^{\text{orig}}(k+1) = M_1^{\text{orig}}$ as its *arrival covariance* (the weight in the quadratic arrival penalty is its inverse). This is extraction of state-estimation covariance information from the original KKT system; it is **not** an MHE weighting-parameter derivative. See [3, Sec. 3.3, Eqs. (21)–(23)] and the supplied forward-DP technical report [4, Secs. 4–5].

The 2008 Section 4 expression

$$\Delta z_{N+1} = K_{N+1}\left(y_{N+1} - \hat{y}_{N+1}\right) \tag{40}$$

is a Kalman-*like state correction* due to the next measurement. Again, it is not a Kalman-filter update of $\partial z^*/\partial\theta$.

6.2 What NeuroMHE computes instead

NeuroMHE constructs *another, auxiliary* quadratic objective in $X_k = \partial z_k^*/\partial\theta$ and $W_k = \partial w_k^*/\partial\theta$. Its P_k^{KF} and C_k are matrices in the *sensitivity solver*, generated from the *differential KKT Hessians*. They are not automatically the statistical error covariances of the original physical estimator; the Hessians may incorporate nonlinear second-order multiplier terms and learned cost parameters.

The initial weight P_a in the original MHE is *supplied by the estimator design or learned parameterization*. NeuroMHE's auxiliary filter initializes its own matrix with P_a^{-1}. This step is not an instruction to update the original arrival-cost weight from the sensitivity-filter matrices.

6.3 A direct explanation from the two objective functions

The different roles are particularly clear if we compare the *quadratic objectives* rather than the appearance of their Kalman recursions. For simplicity, write the original MHE using information-form (inverse-covariance) penalty weights P_a, R_k, and Q_k:

$$\begin{aligned} J_1(x, w; \theta) = \frac{1}{2}\|x_b - \bar{x}_b(\theta)\|^2_{P_a(\theta)} \\ + \frac{1}{2}\sum_{k=b}^{t} \|y_k - h_k(x_k)\|^2_{R_k(\theta)} + \frac{1}{2}\sum_{k=b}^{t-1} \|w_k\|^2_{Q_k(\theta)}, \end{aligned} \tag{41}$$

subject to $x_{k+1} = f_k(x_k, w_k)$. Here $\|v\|^2_M = v^\top M v$, and Q_k^{-1}, R_k^{-1} are the physical process and measurement covariances *only if* the corresponding weights are interpreted as statistically calibrated information matrices. An EKF-type covariance update for the *original state estimator*, under the usual linearization and additive-noise assumptions, employs those original weights:

$$\begin{aligned} C_k^{\text{state}} &= \left[(P_k^{\text{state}})^{-1} + H_k^\top R_k H_k\right]^{-1}, \\ P_{k+1}^{\text{state}} &= F_k C_k^{\text{state}} F_k^\top + G_k Q_k^{-1} G_k^\top. \end{aligned} \tag{42}$$

The *exact original-MHE KKT/Riccati covariance extraction* of Zavala et al. may incorporate nonlinear second-order terms; their Sec. 3.3 recovers the familiar EKF formula *after neglecting* the indicated curvature and cross terms. All these quantities concern the original state-estimation/arrival-cost calculation, not its parameter derivatives.

In contrast, the auxiliary problem of NeuroMHE is defined over the *sensitivity variables* $X_k = \partial x_k^*/\partial\theta$ and $W_k = \partial w_k^*/\partial\theta$. Its cost, NeuroMHE Eq. (15a), is

$$\begin{aligned} J_2(X, W) = \frac{1}{2}\operatorname{tr}\left[(X_b - \bar{X}_b)^\top P_a (X_b - \bar{X}_b)\right] \\ + \sum_{k=b}^{t}\left\{\frac{1}{2}\operatorname{tr}(X_k^\top \bar{L}_k^{xx} X_k) + \operatorname{tr}\left[(L_k^{x\theta})^\top X_k\right]\right\} \\ + \sum_{k=b}^{t-1}\left\{\frac{1}{2}\operatorname{tr}(W_k^\top L_k^{ww} W_k) + \operatorname{tr}(W_k^\top L_k^{wx} X_k) + \operatorname{tr}\left[(L_k^{w\theta})^\top W_k\right]\right\}, \end{aligned} \tag{43}$$

subject to $X_{k+1} = F_k X_k + G_k W_k$. The matrix $\bar{L}_b^{xx} = L_b^{xx} - P_a$ separates the arrival Hessian at the initial stage, and $\bar{L}_k^{xx} = L_k^{xx}$ thereafter.

The simple but critical difference. The *running-cost coefficients* in the original MHE are its specified measurement and process-noise weights R_k, Q_k. In the *auxiliary sensitivity MHE*, they are the *original Lagrangian Hessian blocks* $\bar{L}_k^{xx}, L_k^{ww}, L_k^{xw}$, together with mixed parameter derivatives $L_k^{x\theta}, L_k^{w\theta}$. The *arrival information weight* P_a, however, appears **unchanged** as the quadratic coefficient in both objectives. The two arrival terms are nevertheless different functions: the first penalizes $x_b - \bar{x}_b$; the second penalizes $X_b - \bar{X}_b$.

For example, let $e_k = y_k - h_k(x_k)$, $H_k = \partial h_k/\partial x_k$, and let λ_k denote the multiplier of $x_{k+1} - f_k(x_k, w_k)$. The exact second derivatives of the *original* Lagrangian contain, for the simple separable running cost in (41),

$$\begin{aligned} L_k^{xx} &= H_k^\top R_k H_k - \sum_j (R_k e_k)_j \nabla^2 h_{k,j} - \sum_a \lambda_{k,a} \nabla^2_{xx} f_{k,a}, \\ L_k^{ww} &= Q_k - \sum_a \lambda_{k,a} \nabla^2_{ww} f_{k,a}, \\ L_k^{xw} &= -\sum_a \lambda_{k,a} \nabla^2_{xw} f_{k,a}. \end{aligned} \tag{44}$$

Thus the auxiliary matrices can include measurement-residual curvature, nonlinear-dynamics curvature, and state–noise cross terms. They need not equal the original noise-precision matrices, and they do not automatically represent probabilistic error covariances. In particular, when constructing the sensitivity recursion, NeuroMHE uses $(L_k^{ww})^{-1}$, $\bar{F}_k = F_k - G_k(L_k^{ww})^{-1}L_k^{wx}$, and $S_k = L_k^{xw}(L_k^{ww})^{-1}L_k^{wx} - \bar{L}_k^{xx}$ in place of the simple EKF coefficients.

Why the initial covariance-looking matrix agrees. Because the *arrival quadratic weight itself is shared*, the auxiliary recursion necessarily starts from

$$\boxed{P_b^{\mathrm{KF}} = P_a^{-1}.} \tag{45}$$

This algebraic initialization is *not an EKF update of the physical arrival covariance*. It equals the physical initial covariance only *if* the supplied P_a has actually been chosen as the inverse of that physical covariance. If $P_a(\theta)$ or its center is learned, the sensitivity forcing contains $P_{a,\theta}(x_b^* - \bar{x}_b)$ and $-P_a\bar{X}_b$ as appropriate; the initial matrix P_a^{-1} alone cannot produce these derivatives.

When the two sets of matrices can coincide. In the linear-quadratic case, or with an approximation dropping the nonlinear curvature terms and cross Hessians, $L_k^{ww} = Q_k$, $L_k^{xw} = 0$, and $\bar{L}_k^{xx} = H_k^\top R_k H_k$. With consistent initial conditions, the auxiliary P_k^{KF}, C_k then obey the same *numerical* covariance recursion as (42). Its state variables and affine forcing remain parameter sensitivities, not the physical state estimates. This limited numerical coincidence is entirely consistent with the distinct interpretations.

Source distinction. The common arrival weight and Hessian-defined auxiliary running cost are explicitly in NeuroMHE Eqs. (4a), (13)–(15), (17)–(18), as reviewed in [4, Sec. 10]. The use of original-state covariance to update the arrival cost is fast MHE Sec. 3.3, not an auxiliary sensitivity-filter derivation [3].

6.4 Why the initial inverse arrival weight does not equate the two covariance roles

Let $b = t - N$. The original NeuroMHE objective includes the arrival penalty

$$\Gamma_b(x_b;\theta) = \tfrac{1}{2}(x_b - \bar{x}_b(\theta))^\top P_a(\theta)(x_b - \bar{x}_b(\theta)), \tag{46}$$

with the *supplied* information weight $P_a(\theta) \succ 0$. It may be tuned/learned; the paper does *not* require P_a to be the inverse of an EKF-computed physical estimation-error covariance. If it actually is chosen as $P_a = (\Sigma_b^{\mathrm{orig},-})^{-1}$, then $P_a^{-1} = \Sigma_b^{\mathrm{orig},-}$ at that boundary, but this is a numerical equality conditional on the original estimator's arrival-weight design.

The *auxiliary sensitivity MHE* uses the quadratic term

$$\tfrac{1}{2}\operatorname{tr}\left[(X_b - \bar{X}_b)^\top P_a(X_b - \bar{X}_b)\right], \qquad \bar{X}_b = \frac{\partial \bar{x}_b}{\partial \theta}. \tag{47}$$

Its Kalman-form recursion is initialized in NeuroMHE Eq. (17a) as

$$\boxed{P_b^{\mathrm{KF}} = P_a^{-1}}, \qquad C_b = \left(I - P_b^{\mathrm{KF}} S_b\right)^{-1} P_b^{\mathrm{KF}} = (P_a - S_b)^{-1}. \tag{48}$$

This is a consequence of the *auxiliary problem's quadratic boundary weight*. It does **not** (i) compute the original state estimate, (ii) differentiate the original arrival covariance, or (iii) prescribe the covariance/weight to be used in the next physical MHE window.

For clarity, when P_a itself depends on θ, differentiating the original arrival stationarity contribution produces, for one parameter direction,

$$\frac{\mathrm{d}}{\mathrm{d}\theta}\left[P_a(\theta)(x_b^* - \bar{x}_b(\theta))\right] = P_a(X_b - \bar{X}_b) + P_{a,\theta}(x_b^* - \bar{x}_b). \tag{49}$$

The mixed-derivative forcing $P_{a,\theta}(x_b^* - \bar{x}_b)$ and the inherited center sensitivity $\bar{X}_b$ are distinct from the initial covariance-like coefficient P_a^{-1}. This is precisely why the full parameter-gradient recurrence contains additional affine terms; merely propagating a covariance cannot recover the gradients.

6.5 The different update laws, and a possible special-case numerical match

NeuroMHE Eqs. (18b)–(18c) compute

$$\begin{aligned} P_{k+1}^{\mathrm{KF}} &= \bar{F}_k C_k \bar{F}_k^\top + G_k (L_k^{ww})^{-1} G_k^\top, \\ C_k &= \left((P_k^{\mathrm{KF}})^{-1} - S_k\right)^{-1}, \\ \bar{F}_k &= F_k - G_k (L_k^{ww})^{-1} L_k^{wx}, \\ S_k &= L_k^{xw} (L_k^{ww})^{-1} L_k^{wx} - \bar{L}_k^{xx}. \end{aligned} \tag{50}$$

These are *auxiliary sensitivity-filter coefficients*, evaluated from the full stagewise differential KKT blocks. In contrast, fast MHE Section 3.3 reduces the original estimator's Riccati/KKT covariance quantities to an *EKF-style physical state covariance update* only after discarding selected nonlinear second-order terms and the state–noise cross contributions; see Eqs. (21)–(23) of [3].

Indeed, in a linear-quadratic Gaussian special case with additive process noise, no state–noise Hessian cross terms, and consistent quadratic weights, the coefficients satisfy

$$\bar{F}_k = A_k, \quad L_k^{ww} = Q_{\mathrm{noise},k}^{-1}, \quad S_k = -H_k^\top R_{\mathrm{meas},k}^{-1} H_k. \tag{51}$$

Then (50) becomes the familiar Kalman covariance update:

$$\begin{aligned} C_k &= \left((P_k^{\mathrm{KF}})^{-1} + H_k^\top R_{\mathrm{meas},k}^{-1} H_k\right)^{-1}, \\ P_{k+1}^{\mathrm{KF}} &= A_k C_k A_k^\top + G_k Q_{\mathrm{noise},k} G_k^\top. \end{aligned} \tag{52}$$

Thus, **the numeric matrices need not be different**. They can coincide (even exactly with fast-MHE KKT-derived Π_k, M_k when the same coefficient blocks are used), as established in sections 2 and 4. What differs is the *computational role*: fast MHE Section 3.3 extracts covariance for the original estimator's next arrival penalty, while NeuroMHE uses those curvature-related coefficients together with $L^{x\theta}$, $L^{w\theta}$, and $\bar{X}_b$ to compute *parameter-sensitivity trajectories*. Equal coefficients do not imply equal calculated outputs or the same algorithmic interpretation.

For a general nonlinear estimator, the differential-KKT Hessians also differ from an EKF's Jacobian-only covariance approximation. For example, a scalar measurement term

$$\ell(x) = \tfrac{1}{2} r\,[y - h(x)]^2 \quad \Longrightarrow \quad \ell''(x) = r\,[h'(x)]^2 - r\,[y - h(x)]h''(x). \tag{53}$$

The conventional EKF information update uses the first curvature term $r[h'(x)]^2$, while the exact Lagrangian Hessian entering NeuroMHE additionally contains the residual-curvature term (and may contain dynamics-multiplier curvature and L^{xw} terms). This explains why the numerical recursions *generally differ from the conventional EKF covariance approximation*, even though their Schur-complement algebra may be the same.

Three distinct objects, never to be conflated. (1) Original state-estimation covariance and the next-window arrival weight; (2) P_k^{KF}, C_k, the predicted/filtered *coefficient matrices* of the auxiliary sensitivity solver; and (3) $\hat{X}_{k|k}^{\mathrm{KF}}$ and $\hat{X}_{k|t}$, the actual *MHE parameter-sensitivity matrices*. In particular, $P_b^{\mathrm{KF}} = P_a^{-1}$ does not identify (2) or (3) with (1).

	Original MHE covariance (fast MHE Sec. 3.3)	**Auxiliary gradient filter (NeuroMHE)**
Purpose	Covariance extraction, update of physical MHE arrival cost	Recover gradients of state/noise estimates versus learned parameters
Matrices	Original KKT-derived Π_k, M_k; EKF approximation under extra assumptions	P_k^{KF}, C_k from differential KKT Hessians; possibly numerically equal
Other terms	Original measurements and state-estimate update	Mixed derivatives $L^{x\theta}, L^{w\theta}$ and inherited $\bar{X}_b$
Output	Updated arrival covariance/precision for next original MHE	$\hat{X}_{k\|k}^{\mathrm{KF}}, \hat{X}_{k\|t} = \partial \hat{x}_{k\|t}/\partial\theta$
Explicit gradient KF in 2008?	**No**	**Developed in NeuroMHE**

7. Is the conversion trivial, and what is original?

7.1 Degree of difficulty of each transformation

Transformation	Assessment
Identify matched stage Hessians Q, D, R and A, G	Direct for a matching original differential KKT model
$\Pi_0 \leftrightarrow C_0$ and $M_k \leftrightarrow P_k^{\mathrm{KF}}$	Matrix identities and Schur-complement recognition
General residual $r_M \mapsto$ predicted sensitivity $-r_M$	Additional affine-variable reinterpretation and recursive proof
Original Newton RHS $\mapsto$ MHE weighting-parameter forcing	Must differentiate the specific original MHE objective
Arrival-center sensitivity $-P_a\bar{X}$	Distinct boundary calculation; cannot be dropped
Backward Newton recovery $\mapsto$ filtered sensitivity plus multiplier smoother	Algebraic transformation plus multiplier-index alignment
Explicit auxiliary matrix-valued MHE and equivalence proof	New optimization-level construction, not supplied by renaming the Riccati matrices

Definitive non-triviality and originality assessment. Yes—the conversion is non-trivial as a complete sensitivity algorithm. The easy part is algebraically matching the forward Schur-complement matrices. The non-trivial parts are replacing the Newton residual by the correct parameter-derivative KKT forcing; introducing the transformed filtered/predicted *sensitivity* variables; differentiating the incoming arrival center and learned weight; aligning the backward multipliers and proving the smoothing reconstruction; and *constructing and proving* the auxiliary matrix-valued MHE whose optimal solution is the desired gradient. The fast-MHE paper does not explicitly carry out that construction. Therefore DMHE/NeuroMHE make a **substantive, interpretable methodological contribution to differentiable optimal estimation** relative to Zavala et al. (2008), even though the underlying elimination algebra and $O(N)$ scaling are not novel in isolation.

This conclusion is not a claim that deriving every individual matrix identity is difficult: the *complete optimization-level correspondence and gradient-filter interpretation* are the original intellectual steps. The distinction between auxiliary covariance-like coefficients and original EKF arrival covariance is demonstrated in section 6.

7.2 Prior art versus novel presentation and capability

The scientific claim supported by the two papers is not that DMHE/NeuroMHE invented the Schur complement, Riccati algebra, the Kalman filter, or linear-horizon KKT solution. Fast MHE [3] had already established the appropriate KKT structures, a Riccati decomposition, and general NLP sensitivity analysis.

Specific to DMHE/NeuroMHE is the explicit proof that the relevant derivative trajectories are the optimum of a *matrix-valued auxiliary MHE*, whose solution can be organized as a Kalman-like filtering stage, a backward multiplier stage, and sensitivity recovery. This is substantively more interpretable than treating the derivative solely as an unknown in a KKT linear system. NeuroMHE extends the earlier DMHE development with a more general Lagrangian differential KKT treatment, incoming-prior sensitivity, neural weight adaptation, and extensive physical validation [1, 2].

This is a claim of **originality relative to the specific fast-MHE paper**, not an exhaustive worldwide priority assertion. Differentiable optimal-control formulations including DiffMPC and PDP have earlier

optimization-level counterparts [9, 7]. The estimator-specific correspondence, its proof, and the Kalman sensitivity implementation are the defensible distinguishing features.

8. Full numerical operation counts and wall-clock scope

Let $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{n\times p}$, and $AX = B$, with $n \asymp Nd$ for horizon N, fixed stage dimension d, and p sensitivity directions. All costs below are arithmetic operations and exclude derivative evaluation, problem-definition labor, and cached symbolic setup. They do *not* identify factorization with a Riccati recursion.

8.1 Explicit inverse: what the historical exponent describes

Your thesis writes $X = -A^{-1}B$. If one explicitly forms the generally dense inverse A^{-1}, it must at least output n^2 entries; thus the output work and storage are $\Omega(n^2)$. Conventional dense inverse algorithms use $O(n^3)$ arithmetic. The historical Coppersmith–Winograd-based fast dense linear-algebra bound is $O(n^{2.376})$ (a theoretical historical upper bound, not today's best bound and not an optimal lower bound).

If the inverse is *first materialized* and multiplied with B by conventional arithmetic, the total is

$$T_{A^{-1}B} = O(n^{2.376}) + O(n^2 p) = O(N^{2.376} + N^2 p) \quad \text{for fixed } d. \tag{54}$$

For fixed p, this is $O(N^{2.376})$. The exponent also characterizes theoretical dense linear-system algorithms based on fast matrix multiplication, so it is not unique to explicitly forming an inverse. Formal $A^{-1}B$ notation does not require inverse construction.

8.2 Conventional general-purpose dense LU

With pivoting, $PA = LU$. The full computation has three distinct stages:

$$\begin{aligned} PA = LU: &\quad O(n^3) && \text{numerical factorization,} \\ LY = PB: &\quad O(n^2 p) && \text{forward triangular solve,} \\ UX = Y: &\quad O(n^2 p) && \text{backward triangular solve.} \end{aligned} \tag{55}$$

Thus $T_{\text{dense LU}} = O(n^3 + n^2 p)$, or $O(N^3)$ for fixed d, p. If numerical factors are already cached, only the $O(n^2 p)$ triangular solves are newly required. The correct order is *first* $LY = PB$, *then* $UX = Y$.

8.3 Sparse LU or $LDL^\top$ on bounded-stage chain KKT

With a suitable ordering, bounded fill, and admissible pivot choices, the numerical factorization may be $O(Nd^3)$ and the triangular solves $O(Nd^2 p)$:

$$T_{\text{sparse}} = T_{\text{numerical factorization}} + T_{\text{triangular solves}} = O(Nd^3 + Nd^2 p). \tag{56}$$

This is conditional, not an unconditional guarantee for arbitrary generic sparse pivoting. Numerical factorization time *is included* in the expression. A cached factorization changes incremental right-hand-side costs.

8.4 Classical Riccati versus forward Kalman sensitivity recursion

A direct LQR Riccati algorithm has a backward time-indexed recursion $O(Nd^3)$, then a forward solution recovery $O(Nd^2 p)$, with no independent full-KKT LU factorization inserted before it. NeuroMHE's

analytical solver has its own *forward* Kalman coefficient/sensitivity recursion $O(Nd^3 + Nd^2p)$ followed by *backward* multiplier and sensitivity recovery $O(Nd^2p)$.

$$\boxed{\begin{aligned} T_{\text{Riccati}} &= T_{\text{backward recursion}} + T_{\text{forward recovery}}, \\ T_{\text{NeuroMHE}} &= T_{\text{forward Kalman}} + T_{\text{backward multiplier/recovery}}. \end{aligned}} \tag{57}$$

At fixed d, p, both are $O(N)$. The names describe distinct actual algorithms, although both exploit time-local elimination.

8.5 Consolidated complexity table

Method	**Primary operation**	**Further numerical steps**	**Total, fixed d, p**
Explicit conventional inverse	Dense inversion $O(N^3)$	Dense multiply $O(N^2p)$	$O(N^3)$; full inverse $\Omega(N^2)$ storage
Fast inverse (historical)	Theoretical dense inversion $O(N^{2.376})$	Conventional multiply $O(N^2p)$	$O(N^{2.376})$ for fixed p
Dense LU	Numerical factorization $O(N^3)$	Forward and backward solves $O(N^2p)$	$O(N^3)$ (factorization included)
Ordered sparse LU/$LDL^\top$	Conditional bounded-fill numerical factorization $O(N)$	Triangular solves $O(Np)$	$O(N)$ for fixed p
Classical LQR Riccati	Backward $O(N)$ time recursion	Forward $O(Np)$ recovery	$O(N)$
Generalized equality Riccati	Structured equality-constrained recursion, typically $O(N)$	Stagewise recovery $O(Np)$	$O(N)$ under bounded-stage regularity
NeuroMHE sensitivity KF	Forward filter $O(N)$ for fixed p	Backward multipliers and recovery $O(Np)$	$O(N)$

Wall-clock comparison: A specialized Riccati or Kalman solver frequently has lower overhead than a general-purpose sparse solver on dense-stage LQ examples, but the ordering is *not universal*. Package design, pivoting, batch size, hardware, numerical tolerances, cached KKT factors, and p determine measured time. Vanroye et al. [11] measure *generalized* Riccati versus sparse packages, which cannot be used as a numerical speedup measurement for *classical* Riccati. Fast MHE's specific cubic-bound remarks for some multiple-shooting factorization configurations do not imply all sparse chain factorizations are superlinear.

9. Why Riccati and ordered sparse elimination are algebraically related

Consider the finite-horizon dynamics $x_{k+1} = A_k x_k + B_k u_k$ with quadratic stage costs $\frac{1}{2}x_k^\top Q_k x_k + \frac{1}{2}u_k^\top R_k u_k$, and terminal cost $\frac{1}{2}x_N^\top P_N x_N$ ($R_k \succ 0$). If the eliminated future stages have cost-to-go $\frac{1}{2}x_{k+1}^\top P_{k+1} x_{k+1}$, the stage-$k$ quadratic in (x_k, u_k) has blocks

$$\begin{bmatrix} Q_k + A_k^\top P_{k+1} A_k & A_k^\top P_{k+1} B_k \\ B_k^\top P_{k+1} A_k & R_k + B_k^\top P_{k+1} B_k \end{bmatrix}. \tag{58}$$

Eliminating u_k by its stationarity equation gives the Schur complement

$$\boxed{\begin{aligned} P_k &= Q_k + A_k^\top P_{k+1} A_k \\ &\quad - A_k^\top P_{k+1} B_k (R_k + B_k^\top P_{k+1} B_k)^{-1} B_k^\top P_{k+1} A_k. \end{aligned}} \tag{59}$$

This entire backward time-indexed procedure is the discrete Riccati recursion. A suitable elimination ordering of the full LQR KKT system reproduces these same stage Schur complements. Generic sparse elimination *need not* choose that ordering or generate identical intermediate quantities; indefinite pivoting also matters. The arithmetic connection explains the shared potential $O(N)$ scaling without conflating a sparse LU/LDL factorization stage with the classical Riccati recursion.

10. The methodological merits of auxiliary optimization

10.1 Constructing the sensitivity problem without numerical KKT factorization

For an equality-constrained problem (2), if the reduced quadratic Hessian is appropriately positive definite, the sensitivity variables are the unique solution to

$$\begin{aligned} \min_X \quad & \tfrac{1}{2} \operatorname{tr}(X^\top H X) + \operatorname{tr}(B_{\text{stat}}^\top X), \\ \text{s.t.} \quad & DX + B_{\text{dyn}} = 0. \end{aligned} \tag{60}$$

Grouping Hessian blocks, dynamics Jacobians, and RHS forcing directly from the original differential KKT equations yields a stagewise auxiliary MPC/MHE. The *mathematical construction* does not require numerically decomposing the original KKT matrix. Once the resulting optimization is explicit, a suitable solver can be selected according to its structure. This is the essential connection demonstrated for control in PDP [7], Safe PDP [8], and for optimal estimation in DMHE/NeuroMHE [1, 2].

10.2 Solver choice is a feature, not a new Riccati identity

Auxiliary mathematical class	Available solution class	Caveat
Dynamics-only LQ	Classical finite-horizon Riccati or Kalman; structured QP software	Positive-curvature and invertibility assumptions
Additional stagewise equalities	Equality-constrained Riccati or structured equality QP	Generalization needed beyond standard LQR
Genuine gradient inequalities	Active-set/IPM constrained QP	May need several linear-system solves
Nonquadratic auxiliary model	Suitable nonlinear solver (e.g., SQP/IPOPT)	The resulting problem is no longer a linear sensitivity QP in general

Software such as acados/HPIPM is primarily exposed through structured optimal-control/QP interfaces, not as an arbitrary-matrix *Riccati decomposition* command. A purpose-built Riccati implementation can work directly on stagewise KKT coefficients; the auxiliary formulation is not mathematically compulsory. It is nevertheless a natural, interpretable bridge from implicit derivatives to existing dynamic-optimization software.

10.3 Soft/barrier treatment is not exact hard-constrained differentiation

Safe PDP [8] incorporates original inequalities via logarithmic barriers and additional equalities via quadratic penalties. Its auxiliary problem for the barrier-approximated control problem retains dynamics

only and can use classical Riccati. For finite barrier strength it computes the derivative of the *approximated optimization problem*. Safe PDP's convergence of both the optimizer and derivative to the original hard-constrained ones requires its stated regularity conditions, notably strict complementarity near the reference optimizer.

Introducing a barrier *only in the backward calculation*, without changing the original hard-constrained forward solution consistently, generally supplies a surrogate gradient, not an exact derivative.

10.4 Weakly active constraints yield a genuinely constrained directional sensitivity problem

For a parameter-dependent inequality $g(z, \theta) \le 0$ active at (z^*, θ_0), consider a one-sided direction d and optimizer variation $v(d)$. Perturbed feasibility and a Taylor expansion give

$$\boxed{g_z v(d) + g_\theta d \le 0.} \tag{61}$$

This is derived from first-order feasibility at the active boundary; it is *not* the invalid operation of differentiating both sides of an arbitrary inequality. For a strongly active inequality, with multiplier $\mu_i > 0$, directional complementarity forces equality. For a weakly active one, $g_i = 0$, $\mu_i = 0$, the directional feasibility inequality may remain nontrivial. The sensitivity also satisfies directional dual feasibility and complementary slackness.

Under the appropriate nonlinear-programming sensitivity assumptions, write $\mathcal{A}_+$ for strongly active constraints and $\mathcal{A}_0$ for weakly active constraints. A representative auxiliary QP for a fixed parameter direction d is

$$\boxed{\begin{aligned} \min_{v} \quad & \tfrac{1}{2} v^\top H v + (Md)^\top v, \\ \text{s.t.} \quad & Dv + Ed = 0, \\ & g_{i,z} v + g_{i,\theta} d = 0, \qquad i \in \mathcal{A}_+, \\ & g_{i,z} v + g_{i,\theta} d \le 0, \qquad i \in \mathcal{A}_0. \end{aligned}} \tag{62}$$

Das and Jäschke [6, Thm. 2, Eq. (7)] explicitly formulate such constrained MHE sensitivity QPs and solve them numerically in a pathfollowing algorithm. They do *not* provide an analytical Kalman-filter-based parameter-weight-gradient recursion analogous to DMHE/NeuroMHE.

10.4.1 Worked learned-cost-weight example

Consider the parameterized optimization with θ a cost coefficient:

$$\min_{x \ge 0} \ \tfrac{1}{2}(x-1)^2 + \theta x. \tag{63}$$

Its optimum is $x^*(\theta) = \max(0, 1-\theta)$. At $\theta_0 = 1$, $x^* = 0$ and the multiplier of $-x \le 0$ is zero; the bound is weakly active. For a parameter direction d, the auxiliary directional QP is

$$\boxed{\min_{v \ge 0} \ \tfrac{1}{2} v^2 + dv, \qquad v(d) = \max(0, -d).} \tag{64}$$

Since $v(d)$ is not a linear function of d, a single fixed equality-KKT solve $Av = Bd$ cannot represent both signs of the perturbation. It is the *directional derivative* that this QP computes; an ordinary Jacobian does not exist at the kink.

For a double-sided condition $\ell(\theta) \le g(z, \theta) \le u(\theta)$, the corresponding active-bound directional conditions are

$$\begin{aligned} \text{active lower:} \quad & g_z v + g_\theta d \ge \ell_\theta d, \\ \text{active upper:} \quad & g_z v + g_\theta d \le u_\theta d. \end{aligned} \tag{65}$$

The strongly active case gives equality; the weakly active case can retain the inequality. For an ordinarily differentiable solution defined on an open parameter neighborhood, feasibility in both d and $-d$ forces the derivative of an active constraint to be zero. Thus, genuine inequalities are especially important in *one-sided directional sensitivity* problems.

10.5 Computational consequence for inequality-constrained QPs

A generic sparse LU/LDL factorization or standalone generalized equality-constrained Riccati sweep solves a linear system; it does not, *by itself*, resolve the active-set/complementarity decisions of (62). These algorithms can nonetheless appear as the *inner* linear solvers in a QP algorithm. For bounded-stage structured linear algebra costing $O(N)$ per iteration, a complete constrained algorithm with $I(N)$ iterations has a representative $O(I(N)N)$ main linear-algebra cost; $I(N)$ need not be bounded as N increases. Tuning, warm-starting, and tolerances can improve wall time but do not prove linear complexity or guarantee exact derivatives.

Core methodological merit. Auxiliary optimization offers a systematic representation of the sensitivities as an *optimization problem*, revealing meaningful state/control/noise variables, stagewise dynamics, boundary conditions, and constraints. *Efficient recursive solution* is one possible consequence, while extensibility to constrained optimization and practical access to existing solver interfaces are additional benefits. Its advantage is not that every problem formulated this way is faster than direct KKT linear algebra.

11. Positioning against differentiable-control and constrained-MHE prior art

Work	Established connection/solver	Scope relevant here
Amos et al., DiffMPC (2018) [9]	Differentiate LQR/MPC optimality; structured LQR backward computation	Differentiable control prior art
Jin et al., PDP (2020) [7]	Differential PMP ↔ auxiliary LQ optimal control; Riccati solver	Explicit control-level auxiliary construction
Jin et al., Safe PDP (2021) [8]	Exact active-equality auxiliary LQR and barrier sensitivity approximation	Constrained-control correspondence and theory
Das–Jäschke (2017) [6]	Inequality-constrained directional sensitivity QP for MHE	Weakly active constraints; numerical pathfollowing
Zavala–Laird–Biegler (2008) [3]	Original MHE KKT Riccati decomposition; NLP sensitivity; original arrival covariance	Linear-system and state-covariance prior art; not auxiliary gradient KF
Wang et al., DMHE (2021) [1]	Auxiliary sensitivity-MHE and Kalman-style recursive gradient	Explicit differentiable optimal estimation
Wang et al., NeuroMHE (2024) [2]	Extended matrix-valued sensitivity-MHE, prior sensitivity, adaptive weights and flights	Journal-level methodological/experimental extension
Zhou–Köhler (2026) [12]	Adjoint VJP and block-tridiagonal KKT structure; notes possible Riccati solution	Scales with learned parameter count; no explicit auxiliary gradient-KF recursion in proposition

Novelty statement supported by these sources. DMHE/NeuroMHE do *not* claim to have invented the earliest $O(N)$ solver for MHE sensitivities. Their demonstrated contribution is an auxiliary *matrix-valued*

parameter-sensitivity estimation system, the proved correspondence to differential MHE optimality, and a constructive Kalman filtering–smoothing calculation, applied to learning estimator parameters and robust flight control. The broader possibility of inequality-constrained auxiliary gradients is an *extension motivated by prior work*, not a result already solved by the published standard NeuroMHE Kalman recursion.

12. Final assessment and recommended wording

Recommended scientifically precise claim. *DMHE and NeuroMHE establish that the sensitivities of a nonlinear MHE optimum with respect to estimator parameters can be characterized as the optimal trajectories of an explicitly constructed auxiliary matrix-valued linear-quadratic MHE. They prove this correspondence and derive an analytical Kalman-filter-based gradient recursion, revealing a filtering–smoothing relationship for sensitivity trajectories. These contributions are distinct from earlier Riccati decomposition methods for original MHE KKT/Newton systems, despite a mathematical mapping of the underlying linear-algebra operations. The auxiliary optimization representation also provides a systematic basis for selecting suitable structured solvers and formulating constrained sensitivity extensions.*

What may fairly be claimed: $O(N)$ horizon complexity of the resulting recursive estimator-gradient algorithm; an explicit sensitivity-estimation correspondence; a Kalman filtering/smoothing interpretation of derivative trajectories; solver choice and constrained extensibility of the broader auxiliary framework; and the additional algorithmic and physical validation contributions established by the respective papers.

What should not be claimed: that generic sparse LU can never be $O(N)$; that the first $O(N)$ MHE KKT/sensitivity solve was NeuroMHE; that fast MHE Section 3.3's original estimator covariance is a parameter gradient; that the auxiliary and original KKT covariance coefficients can never share identical numerical values; that the complete matrix arithmetic is fundamentally unrelated to the older Riccati decomposition; or that a finite backward-only barrier is generally the exact derivative of a different hard-constrained forward optimization.

Explicit final originality assessment: Yes. Transforming the 2008 Riccati/Newton recursion into the complete NeuroMHE Kalman-filter-based parameter-gradient algorithm is non-trivial. A matching numerical elimination does *not* supply by itself an auxiliary matrix-valued estimation formulation, parameter-dependent affine forcing, inherited-prior sensitivity, or the filtering–smoothing interpretation of the computed derivatives. DMHE/NeuroMHE explicitly develop and prove these missing ingredients. This supports a **substantive original structural contribution relative to the fast-MHE paper**, without asserting that the Schur-complement operations, Kalman filtering itself, or $O(N)$ KKT solutions were invented anew.

Covariance qualification and cost-level intuition: The original and auxiliary MHE objectives use the *same arrival information weight* P_a, but their running-cost coefficients are respectively the original quadratic weights and the *Lagrangian Hessians* that define the sensitivity QP. This explains $P_b^{\mathrm{KF}} = P_a^{-1}$ while preserving the distinction between the original MHE arrival covariance and the auxiliary Kalman coefficient matrices: initializing the latter with P_a^{-1} does not make the recursion an EKF arrival-cost update. Their *values may coincide* for matched KKT blocks or in a linear-Gaussian reduction; their calculated outputs and interpretation remain distinct. The 2008 paper's Section 3.3 covariance extraction and Section 4 measurement correction are **not** the parameter-gradient Kalman recursion of NeuroMHE.

Practical implementation advantage of NeuroMHE over fast MHE

Addition to the originality assessment, following Section 12.

Assessment. DMHE (2021) and NeuroMHE (2024) provide a clearer and more directly implementable approach for computing MHE *weighting-parameter gradients* than the fast MHE method of Zavala, Laird, and Biegler (2008). The distinguishing feature is the **complete auxiliary estimation formulation and constructive gradient solver**, rather than a different asymptotic order of Riccati arithmetic.

What is specified by each method?

Fast MHE (2008)	DMHE / NeuroMHE (2021, 2024)
Presents a Riccati decomposition of the *original MHE Newton/KKT system*; gives the general parametric NLP sensitivity equation separately.	Directly identifies the matrix-valued derivatives of MHE state and noise trajectories as the variables of an *auxiliary LQ MHE*.
To compute gradients with respect to learned MHE weights, the implementer must specialize the derivative right-hand sides and adapt the affine Riccati and recovery formulas.	Derives the required stagewise Hessian and mixed-parameter forcing terms, including the inherited arrival-center sensitivity.
Does not explicitly present a Kalman-filter-based recursive algorithm for the full *learned-weight sensitivity trajectories*.	Proves the differential-KKT/auxiliary-MHE correspondence and provides a complete forward Kalman recursion, backward multiplier propagation, and sensitivity recovery (Lemmas 1–2; Algorithm 1).

A practical software interface, not just a mathematical reinterpretation

The auxiliary MHE is constructed **directly from local Lagrangian derivatives and linearized dynamics**. It supplies stagewise objective matrices, affine forcing, dynamics, and boundary conditions in the native form of a structured LQ/QP problem. An implementer can either use NeuroMHE's analytical recursion or pass a compatible auxiliary problem to an optimization-oriented solver, such as HPIPM or acados/HPIPM. For additional strongly active gradient equalities, Vanroye et al.'s `gen_riccati` provides a concrete generalized-Riccati option when its equality-constrained LQ assumptions are satisfied.

Structured constrained extensions

Strictly inactive original inequalities impose no gradient restrictions; strongly active inequalities yield gradient equalities. Weakly active inequalities instead give directional feasibility inequalities and complementarity, naturally represented by a constrained auxiliary QP and handled by an inequality-capable optimizer. This is a systematic extension pathway, distinct from the unchanged analytical NeuroMHE recursion.

Practical conclusion. Fast MHE supplies important Riccati and NLP sensitivity foundations. NeuroMHE goes further by explicitly specifying *what gradient problem to solve and how to solve it*, in a form suitable for direct recursive implementation and compatible structured optimization software. Both methods exploit stagewise structure; the added advantage is the fully developed, reusable weighting-gradient computation framework.

Evidence: Zavala et al. (2008), Secs. 3.1–3.2; DMHE (2021), Sec. III; NeuroMHE (2024), Sec. IV, Eqs. (13)–(20), Lemmas 1–2 and Algorithm 1. See this report's Secs. 3–5, 7, 10.2 and 13.9.

13. Practical runtime accounting and solver-interface comparison

Benchmark scope. Compare the *backward MHE sensitivity calculation* for a fixed, already optimized original MHE trajectory and the same parameter directions. Treat the original MHE stage Hessians, dynamics Jacobians and mixed parameter derivatives as *precomputed common inputs*. Exclude their evaluation from both methods; include all subsequent stage-data organization, global assembly (if used), numerical factorization, and sensitivity recovery. This extends the arithmetic discussion in Sec. 8 and the constrained-QP discussion in Sec. 10.

13.1. Direct global sparse LU or LDL$^\top$ sensitivity solve

Write the equality-constrained differential KKT equations in assembled form, with v collecting all stagewise state/noise sensitivities and λ all equality multipliers:

$$\underbrace{\begin{bmatrix} H_N & D_N^\top \\ D_N & 0 \end{bmatrix}}_{K_N} \begin{bmatrix} v \\ \lambda \end{bmatrix} = - \begin{bmatrix} b_N \\ e_N \end{bmatrix}. \tag{66}$$

Here H_N and D_N are built from the *same stage coefficients* used by the auxiliary problem. For multiple shooting, K_N is generally sparse and time-banded after suitable ordering. In a direct global-sparse implementation, the numerical workload is

$$T_{\text{sparse}} = T_{\text{numerical KKT+RHS assembly}} + T_{\text{LU factorization+solve}}. \tag{67}$$

The *assembly* term covers allocating/indexing the sparse format and placing the precomputed stage blocks into the global KKT sparsity pattern. It does *not* include evaluating the derivative blocks. For fixed stage dimension d and bounded temporal coupling, $\operatorname{nnz}(K_N) = O(Nd^2)$; sparse assembly itself can therefore scale as $O(Nd^2)$, not $O(N^2)$, if implemented efficiently. Bounded-fill factorization can also be $O(Nd^3)$, with triangular solves $O(Nd^2p)$ for p right-hand sides. Reuse numerical factors across the p directions rather than refactorizing for each column.

13.2. Auxiliary LQ MHE with analytical Kalman/Riccati recursion

Instead of explicitly constructing K_N, reorganize those coefficients into stagewise sensitivity costs and linearized dynamics:

$$\begin{aligned} \min_{\{X_k, W_k\}} \quad & \sum_{k=0}^{N-1} \left\{ \frac{1}{2} \begin{bmatrix} X_k \\ W_k \end{bmatrix}^\top H_k \begin{bmatrix} X_k \\ W_k \end{bmatrix} + b_k^\top \begin{bmatrix} X_k \\ W_k \end{bmatrix} \right\} + \ell_N(X_N) \\ \text{s.t.} \quad & X_{k+1} = F_k X_k + G_k W_k + c_k. \end{aligned} \tag{68}$$

Here a single parameter direction is shown; matrix-valued NeuroMHE sensitivities stack directions columnwise. The local blocks include the original MHE Lagrangian Hessians and mixed parameter derivatives, plus the auxiliary arrival contribution; no full-horizon KKT matrix is necessary.

$$T_{\text{aux,KF}} = T_{\text{recursive gradient solver}}. \tag{69}$$

At bounded stage dimension, both this recursion and suitably ordered sparse factorization may be $O(N)$ for fixed p. The expected wall-time advantage is avoidance of general-purpose global sparse assembly/factorization overhead, not a claim of a strictly better horizon-complexity order. Local Riccati/Schur-complement factorizations remain part of the recursive computation.

13.2A. Complete computational cost: KKT assembly versus NeuroMHE

Main result. Starting from identical, already evaluated stagewise MHE derivatives, a direct global sparse KKT solve incurs an additional matrix-assembly step. NeuroMHE's auxiliary Kalman recursion operates directly on these local derivatives. This removes real work and memory traffic, but **does not change the leading asymptotic bound** of an optimally ordered sparse solve.

Assumptions and operation counts

Let N denote the number of temporal stages, $d = n_x + n_w$ the local state/noise dimension, and p the number of parameter-sensitivity directions. Assume temporal coupling is local, the stage Hessians/Jacobians and mixed parameter derivatives have already been evaluated, and sparse factorization has bounded fill. One-time symbolic analysis is excluded from both per-sample timings.

Computational operation	Arithmetic complexity
Direct sparse LU/$LDL^\top$: numerical assembly of global KKT matrix	$O(Nd^2)$
Direct sparse LU/$LDL^\top$: numerical factorization	$O(Nd^3)$
Direct sparse LU/$LDL^\top$: triangular solves for p directions	$O(Nd^2p)$
NeuroMHE: forward Kalman coefficient/sensitivity recursion	$O(Nd^3 + Nd^2p)$
NeuroMHE: backward multiplier and sensitivity recovery	$O(Nd^2p)$

The complete workloads are thus bounded by

$$T_{\text{sparse}} = O\left(Nd^2 + Nd^3 + Nd^2p\right),$$
$$T_{\text{NeuroMHE}} = O\left(Nd^3 + Nd^2p\right).$$

Since $d \geq 1$, the additional Nd^2 term is dominated in the asymptotic expression:

$$\boxed{O\left(Nd^2 + Nd^3 + Nd^2p\right) = O\left(Nd^3 + Nd^2p\right).}$$

In particular, for fixed d and p, both methods have $O(N)$ horizon complexity. A conventional *dense* LU factorization of the full KKT matrix, by contrast, has $O((Nd)^3)$ factorization cost and is asymptotically worse in N.

What the extra assembly step establishes

The direct global sparse workflow must allocate or update the full sparse KKT representation and insert its stage blocks, whereas the auxiliary Kalman solver directly consumes the stagewise arrays. The former therefore carries an *additional $O(Nd^2)$ operation and storage-management burden*. This is a structural and implementation advantage of NeuroMHE, not a proof that its total runtime is always smaller: the constants multiplying the shared $O(Nd^3)$ and $O(Nd^2p)$ work depend on the implementations.

The report's measured implementation illustrates the distinction: 45.13 ms for the reported assembly routine plus 0.63 ms for LU and solves, versus 2.59 ms for the complete Kalman recursion. The observed complete-path advantage is approximately 17.7×; it does not imply that the Kalman arithmetic kernel is faster than LU factorization alone. See Sections 13.7–13.9.

Conclusion. *NeuroMHE avoids global KKT assembly and generic sparse factorization by computing the sensitivities recursively from the stagewise auxiliary MHE. This reduces computational stages and can substantially improve total wall time while retaining the same $O(N)$ horizon scaling as a well-ordered sparse direct solver.*

13.3. Weakly active inequalities: a structured auxiliary QP

For a fixed perturbation direction $d\theta$, weakly active original inequalities contribute first-order *directional* feasibility conditions. Under the assumptions discussed in Sec. 10.4, a representative auxiliary problem is

$$\begin{aligned} \min_{\{X_k, W_k\}} \quad & \sum_k \ell_k^{\text{aux}}(X_k, W_k) \\ \text{s.t.} \quad & X_{k+1} = F_k X_k + G_k W_k + c_k, \\ & E_k^+ X_k + D_k^+ W_k = e_k^+, \\ & E_k^0 X_k + D_k^0 W_k \le e_k^0. \end{aligned} \tag{70}$$

The equality rows represent original equalities and strongly active inequalities; the inequality rows represent weakly active constraints. Strictly inactive original inequalities impose no infinitesimal sensitivity restrictions. A single *unconstrained* Kalman sweep or fixed equality-KKT solve cannot generally determine the weakly active complementarity choice. This QP describes a directional derivative, which need not extend to a two-sided Jacobian.

IPOPT via CasADi: user-facing stage syntax versus internal operations

In a CasADi `Opti` or `nlpsol` formulation, the user may define X_k, W_k, the stage costs, dynamics, boundary conditions and inequality rows in a loop. *Only specifying the stage costs is not sufficient*: the dynamics and gradient constraints must also be provided. CasADi organizes the stage variables into global decision and constraint vectors and exposes their sparse derivative structures to IPOPT [I1–I3].

IPOPT is a **primal–dual interior-point NLP method, not SQP**. Its repeated iterations use sparse global Jacobian/Hessian information and solve KKT-type sparse linear systems. Hence, a global KKT matrix is not assembled *by the user*, but sparse KKT construction and linear-system work still arise *inside the solver*. Invoking IPOPT does not eliminate those costs; it moves their management into the solver interface [I1,I2].

An OCP-structured constrained QP solver

The auxiliary problem (70) is already quadratic with affine constraints. A solver such as HPIPM can accept optimal-control QP stage data and apply a structure-exploiting interior-point method with Riccati-based linear algebra [I4]. This *need not* assemble a full global sparse KKT matrix. Nevertheless, the outer inequality-handling algorithm is iterative, and a bound of $O(N)$ per structured linear-system solve does not guarantee $O(N)$ overall runtime if the iteration count depends on N.

13.4. Runtime components by implementation

Implementation	Global KKT handling	Principal solve
Direct sparse LU/LDL$^\top$	Explicit sparse assembly in direct implementation	Factor + triangular solves
NeuroMHE analytical KF	No global KKT assembly	Stagewise recursion and recovery
CasADi + IPOPT	Hidden from user; sparse global linear systems internally	Iterative primal–dual IPM
OCP QP + HPIPM	Stagewise OCP QP interface; global assembly not required	Structured IPM, Riccati kernels

Implementation distinction. An interface that does not ask the user to assemble a KKT matrix is not, by itself, evidence that the numerical algorithm avoids global KKT construction. Conversely, an optimal-control QP solver can preserve stage structure internally even with inequality constraints.

13.5. A fair empirical timing protocol

For an equality-constrained MHE sensitivity problem and a fixed number p of parameter directions, report the following wall-clock decomposition *after identical stage derivative blocks are available*:

$$T_{\text{sparse}}^{\text{meas}} = T_{\text{KKT+RHS assembly}} + T_{\text{LU factorization+solve}} \tag{71}$$

$$T_{\text{aux}}^{\text{meas}} = T_{\text{recursive gradient solver}} \tag{72}$$

Reusable symbolic analysis is excluded from all per-sample timings and reported speedup ratios. For weakly active cases, supplement these timings with a *separate solver comparison*: CasADi/IPOPT's setup and all interior-point iterations versus a structure-exploiting OCP-QP solver's stage setup and iterations. Do not combine a one-pass analytical sensitivity with a multi-iteration QP runtime without reporting the different mathematical/constraint settings.

Recommended experimental controls.

1. **Matched equations.** Use the same original MHE solution, differentiation scheme, exact or approximated Lagrangian Hessians, incoming arrival-center sensitivity, and gradient right-hand sides. Compare computed sensitivities using matched residual/error tolerances.

2. **Matched input-cost boundary.** Both methods receive the same precomputed derivative blocks. Time KKT/RHS assembly and LU factorization plus solve for the sparse method; time the complete recursive gradient solver for NeuroMHE.

3. **Factor reuse and parameter count.** Use one sparse numerical factorization for all p right-hand sides when valid; do not factor anew for each parameter. Report the p values and whether recursion coefficients are shared across columns.

4. **Cold versus warm runs.** Separate initial symbolic sparsity analysis, compilation/graph construction, and cached numerical operations. Use the same hardware, numerical tolerances, warm-up and threading policies; measure each timed component repeatedly.

5. **Horizon scaling.** Vary N with fixed stage dimensions; report both total backward-pass time and component breakdown, plus memory usage and solve accuracy. This tests practical constants in methods that may *both* have linear-horizon arithmetic.

13.6. Interpretation for DMHE and NeuroMHE

Precise practical conclusion. An explicit auxiliary matrix-valued MHE makes the derivative problem available in stagewise estimation form. For its analytically recursive setting, NeuroMHE avoids forming and factorizing a global KKT matrix, and it can therefore avoid their assembly and general-purpose solver overhead. This strengthens a *wall-clock implementation* argument, not a claim that all sparse LU/LDL$^\top$ methods scale worse than $O(N)$. With weakly active constraints, the auxiliary QP retains its natural stagewise description, but a chosen numerical solver may still construct global sparse KKT systems internally. A structured constrained-QP method can instead exploit Riccati-type stagewise linear algebra within an iterative algorithm.

Implementation references for this section

[I1] COIN-OR, *Ipopt: Implementation Details*. `https://coin-or.github.io/Ipopt/IMPL.html`.
[I2] COIN-OR, *Ipopt: Interfacing Your NLP to Ipopt*. `https://coin-or.github.io/Ipopt/INTERFACES`.
[I3] CasADi, *Documentation*, Sec. 9, Opti stack. `https://web.casadi.org/docs/`.
[I4] G. Frison et al., *HPIPM: High-Performance Interior-Point-Method QP and QCQP Solvers*, github.com/giaf/hpipm.

13.7. Simulation evidence and the meaning of symbolic setup

Answer to the benchmark question. For repeated MHE problems with a fixed stage structure and KKT sparsity pattern, *exclude genuinely reusable symbolic graph construction, sparsity analysis and fill-reducing ordering from the steady-state per-sample runtime*; report these separately as one-time setup. Do *not* exclude the numerical insertion of newly computed stage coefficients into a sparse KKT representation, numerical factorization, or triangular solves. If the implementation reconstructs symbolic objects at every call, that is a real expense of that implementation, not a one-time expense that can simply be subtracted without changing the code.

13.7.1. Measurements transcribed from the supplied simulation screenshot

The user supplied terminal output for three consecutive samples of a *general recursive gradient solver* compared with a global sparse-KKT LU implementation. The timings, in milliseconds, are:

Sample	Kalman	KKT assembly	LU + solve	Sparse total	Ratio
994	2.55	45.20	0.63	45.83	17.97×
995	2.60	45.07	0.63	45.70	17.58×
996	2.61	45.11	0.63	45.75	17.53×
Mean	2.59	45.13	0.63	45.76	17.69×

The displayed state-sensitivity maximum absolute difference is approximately 5.68×10^{-14}; the sparse-KKT residual is approximately 4.4×10^{-16} to 8.9×10^{-16}. These results support numerical agreement of the two computations for the displayed samples. The assembly term accounts for approximately 98.6% of the reported sparse-path time. *The screenshot does not reveal whether symbolic graph generation or ordering is included in the 45 ms assembly measurement or in the 0.63 ms factorization-and-solve measurement.*

13.7.2. Three costs that must not be conflated

(i) Symbolic derivative/KKT graph construction. Building CasADi `SX`/`MX` expressions, generating Jacobian/Hessian graphs, creating reusable functions, and constructing a symbolic full-horizon KKT graph can often be done once for a fixed model/horizon. Their initial creation is an offline/setup cost if compiled or cached. Rebuilding them on every MHE step is avoidable overhead but is still part of the current implementation's wall time until eliminated.

(ii) Sparse symbolic factorization and ordering. Determining a fill-reducing permutation and an elimination structure depends primarily on the sparsity pattern, not on each new set of nonzero numerical values. It can be amortized when the pattern stays fixed *and the selected solver exposes a means to reuse the analysis*. A change of horizon, active set, block dimensions or nonzero pattern can invalidate this reuse. Numerical pivoting can still alter the practical factorization work.

(iii) Numerical KKT assembly. Even when indices/column pointers are cached, changing the Lagrangian Hessians and Jacobians requires putting their current numerical entries into the sparse data array. This cost is included in a fair time-per-sample measurement unless the numerical KKT data are already present as an explicitly stated input to both algorithms. Optimized preindexed array updates may be much cheaper than repeated symbolic concatenation or generic sparse block construction.

13.8. Revised warm-start timing and conclusions

13.8.1. Separate initial setup from repeated computation

For a fixed horizon N, identical stage dimensions, constant KKT sparsity pattern and a valid cached symbolic analysis, report the *steady-state* comparison

$$T^{\text{warm}}_{\text{sparse}} = T_{\text{KKT+RHS assembly}} + T_{\text{LU factorization+solve}} \tag{73}$$

$$T^{\text{warm}}_{\text{auxiliary}} = T_{\text{recursive gradient solver}} \tag{74}$$

Symbolic analysis is offline setup only and is excluded from both measured per-sample runtimes. The auxiliary solver's one-time graph/code generation is also excluded from the comparison. Only numerical operations repeated for new stage coefficients are counted in (73) and (74).

13.8.2. What the present comparison establishes

Supported experimental finding. In the supplied simulation, the complete sparse-KKT path takes approximately 45.76 ms, versus 2.59 ms for the recursive Kalman sensitivity solver: an observed 17.7× end-to-end speed ratio for the implementations and samples shown. The sparse method's time is dominated by its reported assembly routine. It is *not yet established* that 45.13 ms is irreducible numerical assembly rather than a mixture of symbolic work, Python object creation, sparse-format conversion and numeric filling.

The reported LU-factorization-plus-solve segment, approximately 0.63 ms, is actually shorter than the 2.59 ms recursive solver. Thus the screenshot *does not establish that the recursive linear-algebra kernel itself is faster than the sparse factorization kernel*. It establishes that the *current complete sparse implementation* is slower. Both observations should be retained; only the full-system timing justifies the measured 17.7× ratio. The 0.63 ms label alone also does not establish that its internal symbolic ordering time was absent.

13.8.3. Recommended implementation checks before reporting an optimized speedup

1. **Audit the 45 ms assembly timer.** Determine whether it includes creation of CasADi symbolic matrices/functions, repeated `jacobian`/`hessian` calls, Python loops allocating sparse subblocks, CSR/CSC conversion, or only numerical insertion into a previously constructed sparsity pattern.
2. **Precompute reusable structures for both methods.** Freeze the KKT sparsity indices and block locations, cache or compile stage derivative functions, and where supported reuse the symbolic elimination analysis. Fill only the nonzero numerical values at each step. Retain the exact same Hessian/Jacobian inputs and derivative right-hand sides.
3. **Audit the LU call.** For example, SciPy `splu` accepts CSC input and performs a factorization with a selectable column permutation (default `COLAMD`) [I5]. A fresh `splu` call should not automatically be described as a cached symbolic-analysis solve. Distinguish factored-matrix reuse for multiple right-hand sides from symbolic-pattern reuse across changing numerical matrices.
4. **Report two valid metrics.** Give the currently observed complete implementation time *and* a separately measured optimized repeated-solve time after reusable setup has genuinely been moved outside the timed loop. Repeat across horizon sizes and parameter counts; report accuracy and nonzero count.

Refined numerical conclusion. The auxiliary Kalman approach mathematically avoids full-horizon KKT assembly and is efficient in the supplied implementation. A generic sparse LU approach can be substantially improved by reusing its pattern and avoiding repeated symbolic processing. The remaining, properly measured numerical-assembly and solve costs determine whether the runtime advantage persists under optimized matched implementations.

[I5] SciPy, *`scipy.sparse.linalg.splu` documentation*, `https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.splu.html`.

13.9 From stagewise derivative blocks to the KKT solve

Precise distinction. For a direct general-purpose sparse LU/LDL$^\top$ sensitivity solve, the already evaluated local blocks are *numerically assembled* into a single sparse KKT coefficient matrix, and the local parameter-derivative forcing terms are stacked into its global right-hand side (RHS). The original MHE's *auxiliary LQ sensitivity system*, when solved with a purpose-built Kalman/Riccati recursion, works directly with the local blocks and does not need either global assembly. Both approaches start from the *same* derivatives; their subsequent data organization and solution algorithms differ.

13.9.1 Common inputs: blocks already available from original MHE differentiation

At an evaluated optimal MHE trajectory, denote the local sensitivity variables by $X_k = \partial x_k^*/\partial\theta$ and $W_k = \partial w_k^*/\partial\theta$. For each nonterminal stage, the supplied second derivatives and dynamics Jacobians are

$$\mathcal{H}_k = \begin{bmatrix} L_k^{xx} & L_k^{xw} \\ L_k^{wx} & L_k^{ww} \end{bmatrix}, \quad q_k = \begin{bmatrix} L_k^{x\theta} \\ L_k^{w\theta} \end{bmatrix}, \quad (F_k, G_k), \qquad k = 0, \dots, N-1. \tag{76}$$

The terminal stage has $\mathcal{H}_N = L_N^{xx}$ and $q_N = L_N^{x\theta}$. The initial L_0^{xx} and q_0 also include the correct arrival-weight and incoming arrival-center sensitivity terms, as described in Secs. 2 and 4. With parameter-independent dynamics, the sensitivity dynamics are $X_{k+1} - F_k X_k - G_k W_k = 0$. Every q_k may contain multiple RHS columns (one per learned parameter). **Evaluating these blocks is common work and is excluded from the matched solver-only benchmark.**

13.9.2 Direct global sparse LU: assemble the coefficient matrix *and* RHS

For a concrete two-transition horizon ($N = 2$), arrange the unknowns as $(X_0, W_0, X_1, W_1, X_2, \Lambda_0, \Lambda_1)$, where Λ_k is the sensitivity multiplier of $X_{k+1} - F_k X_k - G_k W_k = 0$. The global differentiated KKT equations are explicitly

$$\underbrace{\begin{bmatrix} L_0^{xx} & L_0^{xw} & 0 & 0 & 0 & -F_0^\top & 0 \\ L_0^{wx} & L_0^{ww} & 0 & 0 & 0 & -G_0^\top & 0 \\ 0 & 0 & L_1^{xx} & L_1^{xw} & 0 & I & -F_1^\top \\ 0 & 0 & L_1^{wx} & L_1^{ww} & 0 & 0 & -G_1^\top \\ 0 & 0 & 0 & 0 & L_2^{xx} & 0 & I \\ -F_0 & -G_0 & I & 0 & 0 & 0 & 0 \\ 0 & 0 & -F_1 & -G_1 & I & 0 & 0 \end{bmatrix}}_{\text{assembled sparse KKT matrix } \mathcal{K}_2} \underbrace{\begin{bmatrix} X_0 \\ W_0 \\ X_1 \\ W_1 \\ X_2 \\ \Lambda_0 \\ \Lambda_1 \end{bmatrix}}_{\text{global sensitivity unknowns}} = - \underbrace{\begin{bmatrix} L_0^{x\theta} \\ L_0^{w\theta} \\ L_1^{x\theta} \\ L_1^{w\theta} \\ L_2^{x\theta} \\ 0 \\ 0 \end{bmatrix}}_{\text{assembled global RHS}} . \tag{77}$$

The two $-F_k, -G_k, I$ rows come from the dynamics Jacobians. The diagonal/cross Hessian blocks come from $L_k^{xx}, L_k^{xw}, L_k^{wx}, L_k^{ww}$. The transpose blocks are the equality-multiplier couplings. The numerical RHS contains the mixed derivatives and any inherited arrival sensitivity. For an arbitrary N, the identical stagewise pattern expands to a larger sparse saddle-point matrix, not generally a dense one.

Actual direct sparse implementation: insert precomputed numerical blocks into a global CSC/CSR or equivalent sparse layout; form the global RHS; numerically factor the resulting KKT matrix; perform forward/backward triangular solves for all requested parameter directions; the resulting solution vector contains X_k, W_k. The global numerical insertion/stacking costs are incurred even when the matrix sparsity pattern and symbolic ordering have been cached (unless the global numerical data already exist and that reuse is explicitly part of the benchmark).

13.9 Stagewise KKT assembly versus the auxiliary recursion (continued)

13.9.3 Auxiliary LQ MHE: form local costs and apply a recursive solver

The *same* $\mathcal{H}_k, q_k, F_k, G_k$ define the auxiliary MHE directly, without a global $\mathcal{K}_N$ or stacked RHS. For one parameter direction, writing $\xi_k = [X_k^\top, W_k^\top]^\top$, its quadratic program is

$$\begin{aligned} \min_{\{X_k, W_k\}} \quad & \sum_{k=0}^{N-1} \left(\tfrac{1}{2} \xi_k^\top \mathcal{H}_k \xi_k + q_k^\top \xi_k \right) + \tfrac{1}{2} X_N^\top L_N^{xx} X_N + (L_N^{x\theta})^\top X_N, \\ \text{subject to} \quad & X_{k+1} = F_k X_k + G_k W_k, \qquad k = 0, \ldots, N-1. \end{aligned} \tag{78}$$

With multiple sensitivity directions, the equivalent matrix-valued cost uses traces and columns of q_k. NeuroMHE's analytical solver directly applies its forward Kalman recursion and backward multiplier/sensitivity recovery using these *stagewise* arrays. There is **no numerical assembly of a global sparse KKT matrix or global linear-system RHS**. This specifically describes the analytical recursive implementation, not every solver to which one might submit the auxiliary problem.

13.9.4 Match exactly what must be timed after the blocks are given

For the same precomputed stagewise blocks $F_k, G_k, L_k^{xx}, L_k^{xw}, L_k^{ww}, L_k^{x\theta}, L_k^{w\theta}$, compare the operations performed on each solve:

$$\begin{aligned} T_{\text{sparse}} &= T_{\text{global KKT+RHS assembly}} + T_{\text{LU factorization+solve}}, \\ T_{\text{aux}} &= T_{\text{recursive gradient solver}}. \end{aligned} \tag{79}$$

The sparse RHS may cost relatively little to stack; matrix assembly is normally the more substantive data-organization step. For a fixed-size stage, efficient preindexed sparse KKT assembly has $O(N)$ nonzeros and *can itself be* $O(N)$; it is not mathematically required to take tens of milliseconds. Graph construction, symbolic factorization/ordering, and any repeated CasADi symbolic processing should be separately audited rather than silently labeled numerical assembly. Both implementations may also have reusable one-time setup.

13.9.5 Interpretation of the supplied simulation and constrained extensions

Direct sparse KKT		Auxiliary Kalman	
Global KKT assembly (reported)	45.13 ms	Stagewise recursive solve (total)	2.59 ms
Numerical LU + solves (reported)	0.63 ms	No assembled global KKT/RHS	–
Total sparse implementation	45.76 ms	Observed complete runtime ratio	17.7×

What this supports. For the displayed samples, the proposed recursive gradient implementation avoids the measured expensive global assembly path and achieves roughly 17.7× lower total runtime, with near-machine-precision agreement. The 0.63 ms sparse factorization/solve is *faster* than the 2.59 ms recursive path in isolation. Therefore the measured advantage is for *the complete implementations*, not evidence that the Kalman linear algebra kernel is universally faster. Profiling must determine how much of the reported 45.13 ms is essential numerical insertion and how much is avoidable Python/symbolic construction or format conversion.

Constraint scope. Strongly active inequalities can be carried as gradient equalities in a suitably generalized recursive scheme. Weakly active constraints lead to a directional auxiliary QP with inequalities. If that QP is passed to IPOPT/CasADi, the user need not construct a global KKT system explicitly, but IPOPT generally builds sparse global linear systems internally. A structure-exploiting optimal-control QP solver may instead use stagewise Riccati-based inner solves. **The no-global-assembly claim applies without qualification to the explicit NeuroMHE analytical recursion, not to every generic numerical solver of an auxiliary QP.**

Source documents and references

References

[1] B. Wang, Z. Ma, S. Lai, L. Zhao, and T. H. Lee, "Differentiable Moving Horizon Estimation for Robust Flight Control," *Proceedings of IEEE CDC*, 2021. Source: user-supplied PDF; Lemmas 1–2 and the proposed Kalman gradient recursion.

[2] B. Wang, Z. Ma, S. Lai, and L. Zhao, "Neural Moving Horizon Estimation for Robust Flight Control," *IEEE Transactions on Robotics*, vol. 40, pp. 639–659, 2024. DOI: 10.1109/TRO.2023.3331064. Source: user-supplied PDF, Sec. IV, Eqs. (13)–(20), Lemmas 1–2, Apps. A–C.

[3] V. M. Zavala, C. D. Laird, and L. T. Biegler, "A fast moving horizon estimation algorithm based on nonlinear programming sensitivity," *Journal of Process Control*, vol. 18, pp. 876–884, 2008. Source: user-supplied PDF, Secs. 3.1–3.3 and 4, Eqs. (9)–(25). *Original MHE covariance in Sec. 3.3, not parameter-gradient Kalman filtering.*

[4] "Moving Horizon Estimation, Forward DP, and Riccati Arrival Costs," technical notes supplied in this conversation (19 October 2026), particularly Secs. 4–5, 7, and 10. This is a *secondary derivation*, not a substitute for the source papers.

[5] H. Cox, *Estimation of State Variables for Noisy Dynamic Systems*, D.Sc. thesis, Massachusetts Institute of Technology, 1963. Source: user-supplied thesis; Ch. II, Eqs. (2.20)–(2.22), (2.50)–(2.55).

[6] T. Das and J. Jäschke, "Pathfollowing approach for a sensitivity based moving horizon estimator," *Proceedings of IEEE CDC*, 2017. Source: user-supplied PDF, Thm. 2, Eq. (7), Alg. 1, and Sec. VI.

[7] W. Jin, Z. Wang, Z. Yang, and S. Mou, "Pontryagin Differentiable Programming: An End-to-End Learning and Control Framework," *NeurIPS*, 2020. Source: user-supplied PDF; Sec. 5, Lemmas 5.1–5.2.

[8] W. Jin, S. Mou, and G. J. Pappas, "Safe Pontryagin Differentiable Programming," *NeurIPS*, 2021. Source: user-supplied PDF; Theorems 1–2, Eqs. (3)–(6), App. E.

[9] B. Amos, I. Jimenez, J. Sacks, B. Boots, and J. Z. Kolter, "Differentiable MPC for End-to-End Planning and Control," *NeurIPS*, 2018. Source: user-supplied PDF.

[10] M. Diehl, H. J. Ferreau, and N. Haverbeke, "Efficient Numerical Methods for Nonlinear MPC and Moving Horizon Estimation," in *Nonlinear Model Predictive Control: Towards New Challenging Applications*, 2009, pp. 391–417. Source: user-supplied e-book, Sec. 4.4 and Eq. (26); sparse direct and Riccati can both attain linear horizon scaling under structure.

[11] L. Vanroye, J. De Schutter, and W. Decré, "A generalization of the Riccati recursion for equality-constrained linear quadratic optimal control," *Optimal Control Applications and Methods*, 2024. Source: user-supplied paper; generalized equality constraints and sparse-solver benchmarks.

[12] Y. Zhou and J. Köhler, "Scalable Identification of Physics-Based and Neural Dynamics from Partial Measurements," arXiv:2610.16559, October 2026. Source: user-supplied preprint, Props. 1–2.

[13] D. Coppersmith and S. Winograd, "Matrix multiplication via arithmetic progressions," *Journal of Symbolic Computation*, vol. 9, pp. 251–280, 1990. Historical fast-matrix-multiplication exponent; the bound is neither a current-best exponent nor a universal lower bound for structured linear solves.

Wang Bingheng